

ASTRA LiveOps Server

.NET 10 / Orleans 기반 수집형 RPG LiveOps 서버 - 아키텍처와 검증

프로젝트 요약

이 문서는 수집형 RPG의 가차 정합성, 콘텐츠 배포, 운영 사고 복구를 구현한 LiveOps 서버의 아키텍처와 검증 결과를 정리한다.
기능 수보다 데이터 정합성, 장애 복구, 운영 가시성과 재현 가능한 검증에 초점을 둔다.

핵심 구현

영역	구현
명령 경계	PlayerAccountGrain 기반 플레이어별 직렬화
영속 기준	PostgreSQL transaction, ledger, audit, completed response
운영 복구	immutable content snapshot, rollback, 대상자 Mail 보상
전송/운영	ASP.NET Core HTTP, TCP+Protobuf, Blazor Admin, Outbox

검증 요약

- 표준 테스트 91건과 PostgreSQL 테스트 17건 통과
- PostgreSQL ADO.NET membership 기반 2-Silo 및 HTTP/TCP E2E 4건 통과
- pool saturation, Outbox retry/dead-letter와 경보를 실제 장애 주입으로 재현
- 한 명령 데모로 가차 replay, 사고 대상 snapshot과 중복 없는 보상을 검증

ASTRA LiveOps Server - 프로젝트 개요

1. 프로젝트 정의

ASTRA LiveOps Server는 트릭컬 리바이브 같은 수집형 RPG의 실제 운영 문제를 기준으로 설계하고 구현한 .NET 10 기반 LiveOps 서버다.

핵심 목표는 기능 수를 늘리는 것이 아니라, C#, Microsoft Orleans, Blazor, PostgreSQL, Redis, TCP와 운영 인프라를 데이터 정합성 및 사고 복구 문제에 일관되게 적용하는 것이다.

2. 최종 한 문장

가차 정합성과 운영 사고 복구를 재현 가능한 테스트와 운영 증빙으로 검증하는 Orleans 기반 수집형 RPG LiveOps 서버.

3. 왜 트릭컬 리바이브 기준인가

트릭컬 리바이브는 캐릭터 수집, 픽업 모집, 재화, 천장, 중복 보상, 이벤트, 출석, 우편, 시즌성 보상, 운영 공지와 보상 지급이 반복되는 LiveOps형 수집 RPG다.

이 구조에서 서버가 증명해야 하는 능력은 단순 CRUD가 아니다.

- 같은 요청이 여러 번 들어와도 재화와 보상이 중복 처리되지 않아야 한다.
- 픽업, 보상표, 이벤트 기간 같은 콘텐츠 설정이 운영 도구에서 안전하게 배포되어야 한다.
- 잘못된 설정이 잠깐 활성화되어도 영향 유저를 추적하고 보상 또는 복구할 수 있어야 한다.
- 대규모 이벤트 직후에도 콘텐츠 조회, 우편 보상과 로그 수집이 병목이 되지 않아야 한다.

4. 구현 범위

실제 코드와 데모는 두 시나리오에 집중한다.

4.1 가차/천장/재화/인벤 정합성

- 재화 차감
- 가차 결과 생성
- 캐릭터 또는 아이템 지급
- 중복 캐릭터 보상
- 천장 포인트 갱신
- 원장과 감사 로그 기록
- Idempotency-Key 기반 중복 요청 방어
- 장애 주입 테스트

4.2 운영 실수 복구/보상

- 잘못된 픽업 또는 보상표 배포
- 처리 당시 content version/checksum 기록
- 영향 유저 산정
- 대상자 snapshot 생성
- 대상 snapshot 기반 Incident Mail 생성
- 수령 시 idempotent 보상 적용
- 복구 결과 추적

5. 명시적 제외 범위

다음 범위는 현재 구현에서 제외하거나 설계 확장 후보로만 둔다.

- 모든 게임 콘텐츠 완성
- 실시간 전투 판정 서버
- 폴스케일 MSA
- 2PC 또는 Orleans Transactions 중심 설계
- CDC/WAL 기반 Outbox 고도화
- 운영 중인 실제 AKS 프로덕션 클러스터

6. 성공 기준

- 같은 Idempotency-Key 100회 재전송 시 실제 가차 처리 1회
- 같은 유저의 동시 재화 사용 요청에서 음수 잔액 0건
- 커밋 후 응답 전 장애 발생 시 재시도에서 같은 response_body 반환
- 잘못된 content version으로 처리된 유저 목록 산정 가능
- Incident Mail 대상 claim이 Redis 장애 시에도 DB fallback으로 동작
- TCP Gateway와 HTTP API가 같은 domain command를 호출
- Blazor Admin에서 draft validation, checksum, publish, rollback, compensation을 시연 가능
- traces/logs/metrics가 API 또는 TCP에서 Grain, DB, Outbox까지 이어짐
- OTLP -> EDOT Collector -> OTel-native Elasticsearch -> Kibana 경로가 실제 동작
- 2-slot PostgreSQL 풀 포화에서 4회 획득 시도, timeout 1회, 회복 성공을 재현
- valid/invalid Outbox 이벤트로 published 1, retry 1, dead-letter 1과 경보 발화를 재현

7. 참고 근거

- 트릭컬 리바이브 공식/스토어/커뮤니티/이벤트 공지
- Microsoft Orleans 공식 문서

문제 정의, 솔루션, 기대효과

1. 문제 정의

수집형 RPG LiveOps 서버의 핵심 과제는 기능 개수보다 정합성, 운영 안정성과 복구 가능성에 있다.

2. 핵심 문제와 솔루션

문제	위험	ASTRA 솔루션
네트워크 재시도	가차/보상 중복 지급	Idempotency-Key, request hash, response_body TTL
동시 요청	재화 음수, 천장 꼬임	PlayerAccountGrain 단위 명령 직렬화
DB 장애	부분 차감/부분 지급	단일 DB transaction, ledger, audit
잘못된 콘텐츠 배포	잘못된 보상/확률 적용	content version/checksum, rollback, incident
대량 보상	대상자 판정 병목	PostgreSQL target snapshot, Redis membership cache
설정 hot spot	이벤트 오픈 직후 지연	ActiveContentCache, EventConfigGrain control plane
관측 불가	장애 원인 추적 실패	OpenTelemetry, EDOT, Elasticsearch/Kibana, correlation id
설계와 구현 불일치	설명만 있고 재현 근거가 없음	두 핵심 시나리오를 테스트와 데모로 검증

3. 내 솔루션

3.1 PlayerAccountGrain

플레이어의 상태 변경 명령은 PlayerAccountGrain을 통과한다. 동일 플레이어의 가차, 보상 수령, 재화 사용 요청을 한 actor queue에서 순차 처리해 race condition을 줄인다. 쓰기 command는 reentrant/interleaving을 허용하지 않는다.

Orleans가 모든 정합성을 해결한다고 보지 않는다. Grain은 실행 경계와 직렬화 경계이고, 최종 원장과 감사, 복구 기준은 PostgreSQL이다.

3.2 PostgreSQL Source of Truth

PostgreSQL은 다음 데이터를 영속 기준으로 가진다.

- players
- wallet_balances
- inventory_items
- owned_characters
- pity_states
- gacha_draw_history
- ledger_entries
- admin_operation_audit
- idempotency_requests
- content_snapshots
- active_content
- mail_definitions
- mail_targets
- mail_claims
- outbox_events

- operational_event_deliveries

3.3 Redis

Redis는 원본이 아니라 가속 계층이다.

- mail target membership cache
- target snapshot ready marker와 TTL

Redis 장애 시 핵심 정합성은 PostgreSQL fallback으로 유지한다.

3.4 Blazor LiveOps Admin

운영 도구는 단순 CRUD가 아니라 콘텐츠 제어와 사고 대응을 지원한다.

- content draft input
- validation result
- active checksum 확인
- rollback
- incident impact query
- compensation creation
- Incident Mail claim replay
- Outbox/dead-letter monitoring
- audit trail
- OIDC authorization code + PKCE 로그인과 역할 기반 BFF 세션

개발 환경은 loopback 전용 토큰 교환을 사용한다. 운영 환경은 외부 OIDC authority가 발급한 사용자별 API access token을 Admin 서버 메모리에만 보관하고, 브라우저에는 HttpOnly 세션 쿠키만 전달한다. 공개 HTTPS origin을 별도 고정해 Ingress TLS 종료 뒤에도 callback URI가 내부 HTTP 주소로 변질되지 않게 한다. 현재 token store는 pod-local이므로 OIDC Admin은 1 replica로 제한하며, 실제 IdP tenant 연동은 자격증명 경계 밖이다.

3.5 TCP Gateway

TCP Gateway는 HTTP API 뒤에 붙지 않는다.

최종 구조:

```
HTTP Client/Admin -> ASP.NET Core API -> Orleans Client -> Grain
TCP Client      -> TCP Gateway  -> Orleans Client -> Grain
```

HTTP와 TCP는 transport만 다르며 같은 command contract, validation, idempotency policy, domain handler를 공유한다.

4. 기대효과

4.1 유저 관점

- 재시도와 네트워크 끊김에도 중복 차감/중복 지급이 없다.
- 장애 이후 같은 요청을 다시 보내면 같은 결과를 받을 수 있다.
- 운영 보상이 누락되거나 중복 수령되지 않는다.

4.2 운영자 관점

- 픽업/이벤트/보상표를 코드 배포 없이 등록하고 검증한다.
- 잘못된 콘텐츠 배포 후 영향 유저를 추적한다.
- 사고 보상은 Incident Mail과 대상 snapshot으로 처리한다.
- 모든 운영 명령은 audit trail로 남는다.

4.3 개발자 관점

- 상태 변경 경계가 PlayerAccountGrain으로 명확하다.

- DB transaction과 outbox로 장애 복구가 설명 가능하다.
- 테스트가 핵심 리스크를 직접 검증한다.
- fault injection 결과가 dashboard, alert, JSON evidence로 남는다.
- 각 기술 선택의 역할과 검증 근거가 명확하다.

4.4 기술 구성과 구현 근거

기술	구현 근거
.NET 10 / C#	ASP.NET Core, Orleans Silo, Worker, TCP Gateway
Microsoft Orleans	PlayerAccountGrain, EventConfigGrain, MailboxGrain
Blazor	LiveOps Admin, OIDC code flow + PKCE BFF 인증 경계
PostgreSQL	source of truth, ledger, audit, idempotency
Redis	incident mail target membership cache와 PostgreSQL fallback
TCP	Protobuf packet gateway
Elastic APM / ELK	EDOT gateway, OTel data stream, dashboard, alert
GitHub Actions	PostgreSQL E2E, ADO.NET Orleans E2E, Docker 5종 build, IaC 검증
Kubernetes / Helm	11-resource chart, migration hook, probes, limits, Secret references
Azure	Terraform 기반 private AKS, ACR, VNet, Log Analytics, private PostgreSQL
Terraform	Azure foundation, azurerm 4.80 validate 통과
Pulumi	기존 AKS에 Helm release를 배포하는 C# program, build와 offline preview 통과

종합 파이프라인 및 흐름도

아래 흐름도는 런타임 요청, 정합성, 콘텐츠 배포, 사고 복구, 비동기 처리와 관측 경계를 같은 시각 규칙으로 정리한다.

1. 전체 아키텍처

2. 각 단의 책임

단	책임
Game Client	HTTP 또는 TCP로 game command 요청
Blazor Admin	콘텐츠 등록, 검증, 배포, 롤백, 보상 생성
ASP.NET Core API	인증, 권한, HTTP validation, Orleans 호출
TCP Gateway	Protobuf framing, session, request id, Orleans 직접 호출
Orleans Client	API/Gateway에서 Silo로 RPC 연결
Orleans Cluster	actor activation, placement, queue, lifecycle
PlayerAccountGrain	플레이어 변경 명령 직렬화
EventConfigGrain	content publish/rollback control plane
ActiveContentCache	hot path content read snapshot
PostgreSQL	최종 상태, 원장, 감사, 멍등성, 대상 snapshot
Redis	사고 우편 대상 membership cache
Outbox Worker	외부 이벤트 처리, retry, poison 분리
Elastic APM/ELK	trace, log, metric 관측

3. 가차 처리 흐름

핵심: Idempotency 요청에는 PENDING 상태를 두지 않는다. 완료 결과만 같은 transaction에 저장한다.

4. Idempotency 최종 흐름

1. API/TCP가 player_id, idempotency_key, request_hash를 command에 포함
2. PlayerAccountGrain이 동일 player command를 순차 처리. 쓰기 command는 reentrant/interleaving 금지
3. DB에서 completed idempotency row 조회
4. 같은 key + 같은 hash + completed가 있으면 response_body 반환
5. 같은 key + 다른 hash면 IDEMPOTENCY_CONFLICT
6. 없으면 도메인 로직 수행
7. 단일 DB transaction 안에서 상태 변경, ledger, audit, outbox, completed response_body 저장
8. commit 전 실패는 전체 rollback
9. commit 후 응답 전 장애는 재시도 시 completed response_body 반환

5. 콘텐츠 배포 흐름

검증 항목:

- 이벤트 기간 overlap
- KST/UTC 변환

- 보상 수량 음수 금지
- 픽업 배너와 가차 pool 일치
- 천장 대상과 보상 table 일치
- publish 전 checksum 생성

6. 운영 사고 보상 흐름

두 종류를 구분한다.

- 전체 공지성 보상: global_mail_definition + claim table
- 사고 대상자 보상: PostgreSQL mail_targets snapshot + Redis membership cache

7. Outbox 흐름

Outbox의 PENDING은 유지한다. 제거 대상은 요청 idempotency PENDING뿐이다.

8. 관측성 흐름

실제 수집 경로:

```
API / TCP / Silo / Worker
-> OTLP/gRPC
-> EDOT Collector gateway
-> OTEL-native Elasticsearch data streams
-> Kibana dashboard / alert
```

필수 correlation fields:

- trace_id
- span_id
- player_id
- command_id
- idempotency_key_hash
- content_version
- grain_type
- grain_id
- db_transaction_id
- outbox_message_id

현재 dashboard가 직접 보여주는 항목:

- 서비스 trace와 metric 수집량
- PostgreSQL connection acquisition failure
- 99.9% acquisition SLO burn rate
- Outbox published, retry, dead-letter

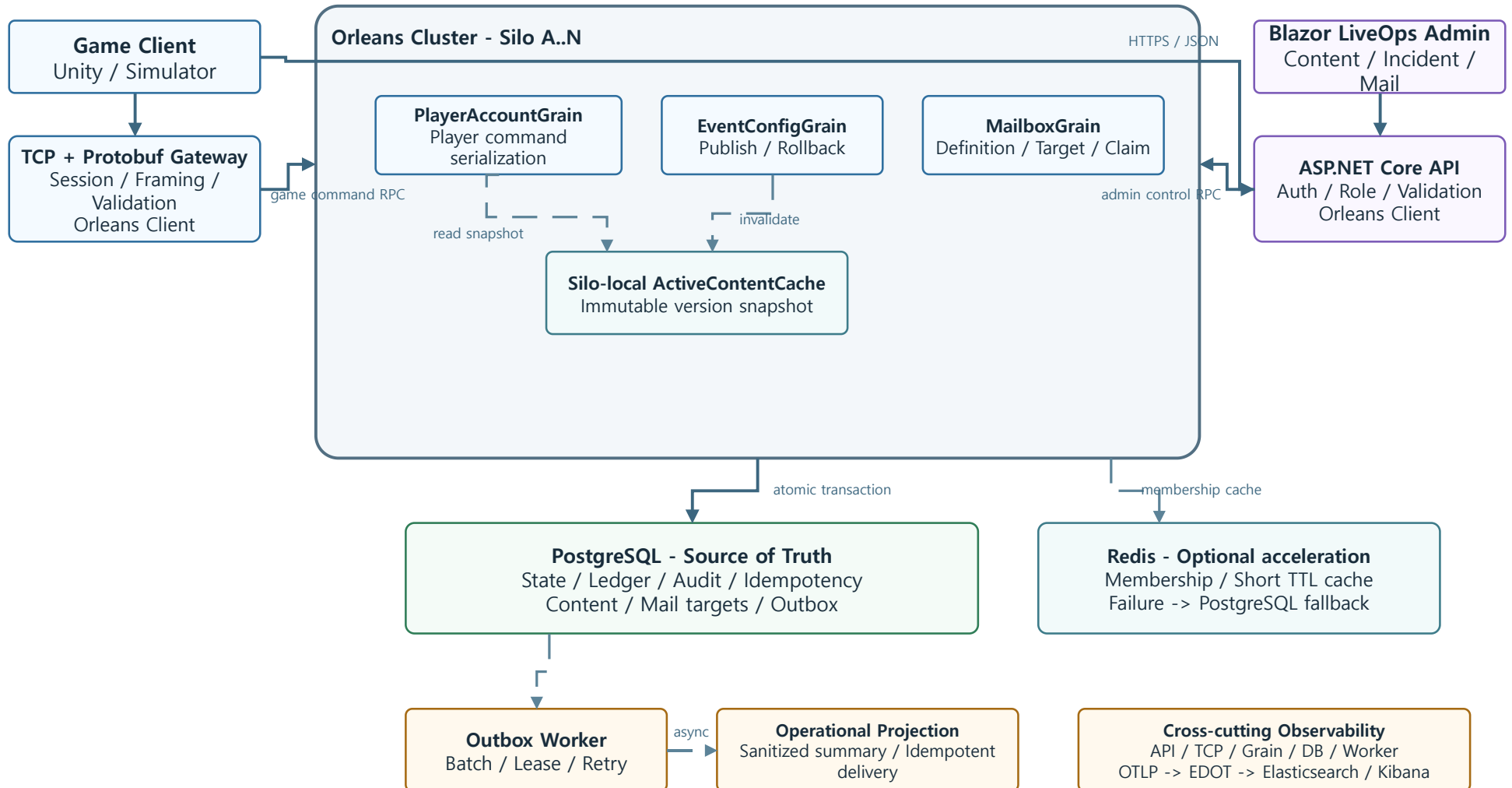
재현 스크립트는 2-slot 풀을 실제 포화시켜 **attempts=4, timeouts=1, burn_rate=250x**, 회복 query 성공을 검증한다. 이어 valid/invalid Outbox 이벤트를 함께 주입해 **published=1, retry=1, dead_letter=1**과 Critical alert 발화를 검증한다.

9. 시각 흐름도 모음

아래 흐름도는 앞선 설계 설명을 실행 순서와 장애 경계 기준으로 다시 정리한 최종 시각 산출물이다.

8.1 전체 구성

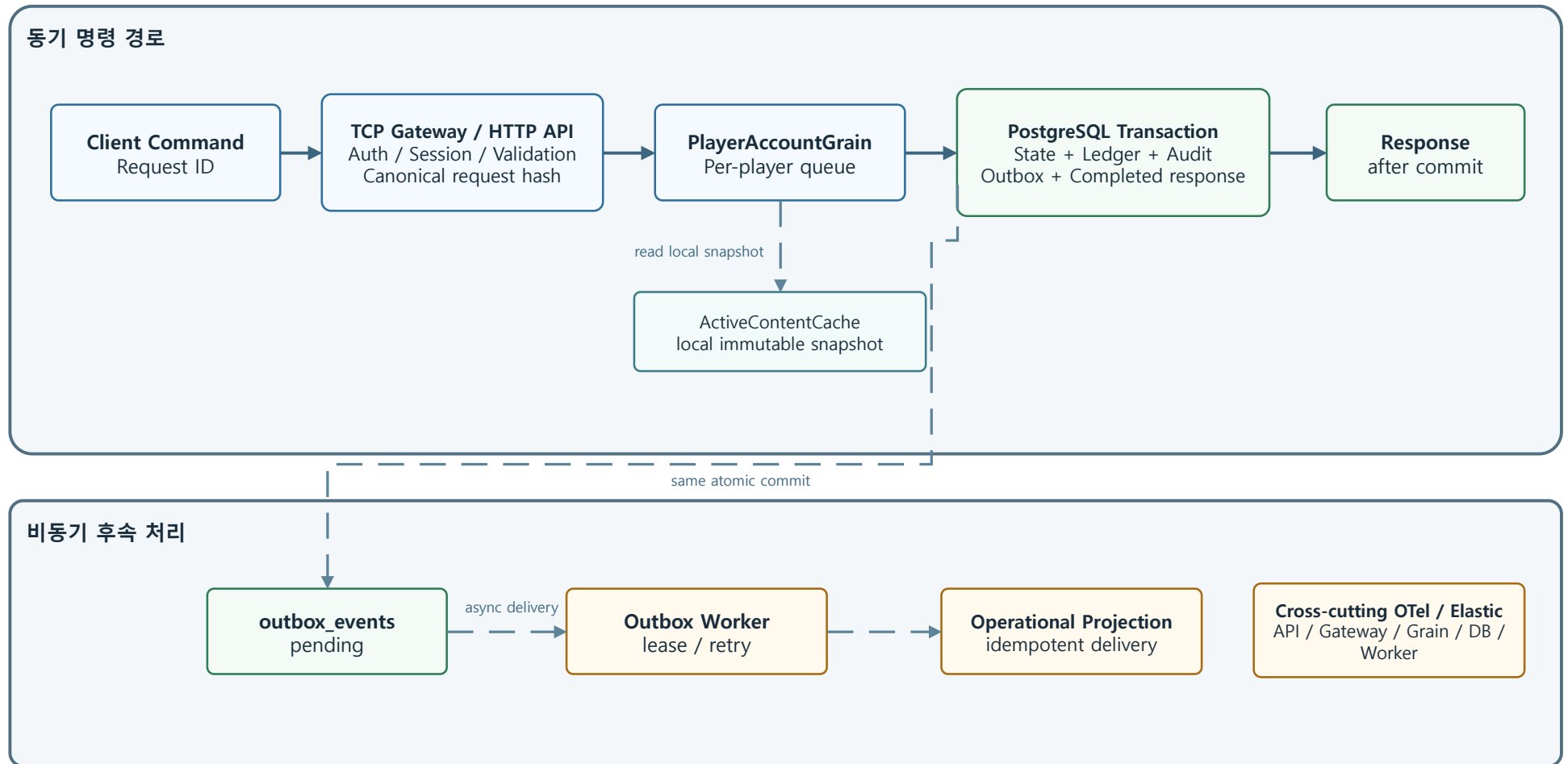
게임 명령 경로와 운영 제어 경로는 분리하고, PostgreSQL commit 이후 비동기 처리를 분기한다.



Solid: command / transaction | Dashed: cache / async | Observability is cross-cutting, not an Outbox destination

8.2 동기 요청과 비동기 분기

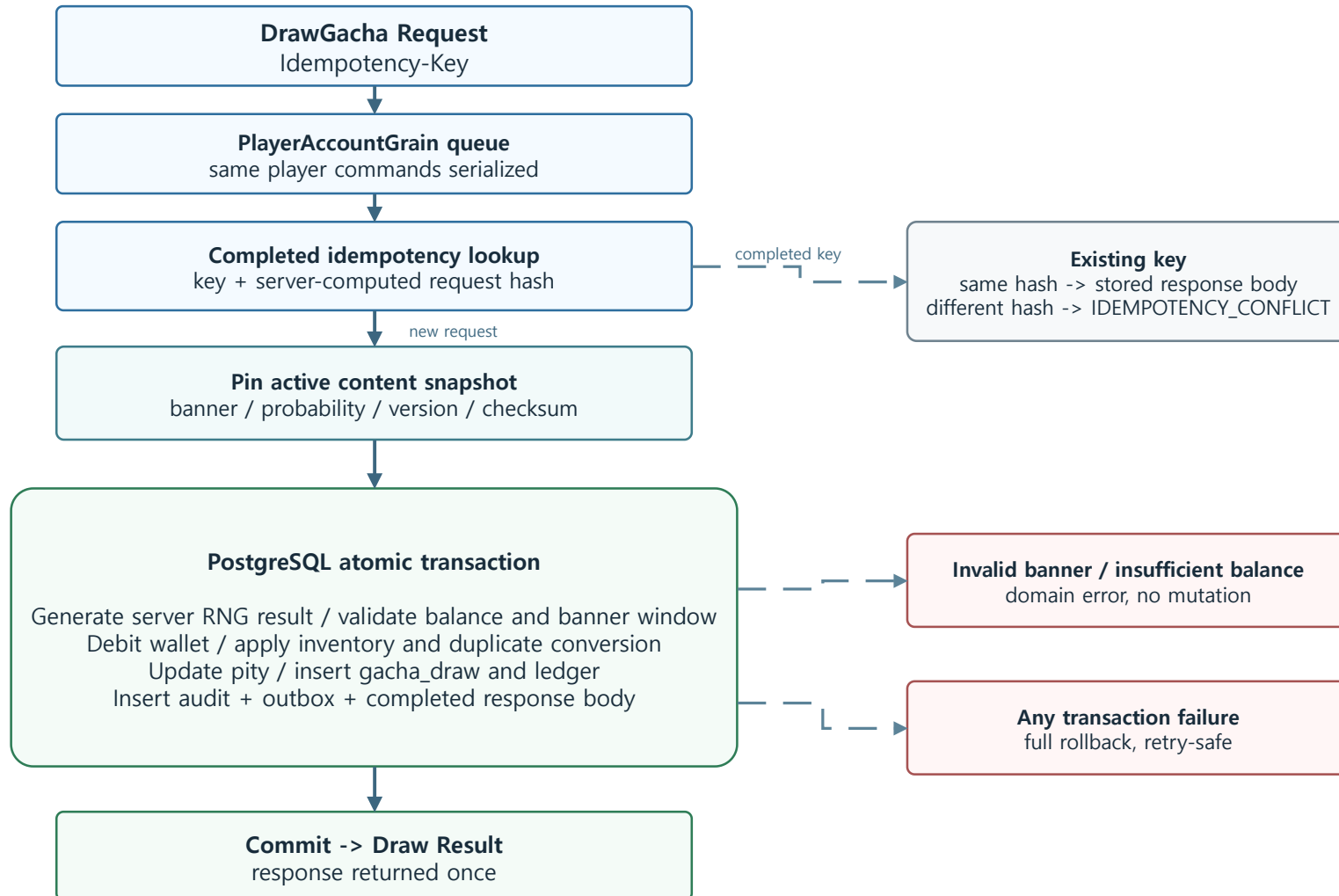
Worker와 Observability를 동기 요청의 직렬 단계로 오해하지 않도록 실행 경계를 분리한다.



The synchronous response path ends at PostgreSQL commit; asynchronous delivery failures do not roll back player state.

8.3 가차 원자 트랜잭션

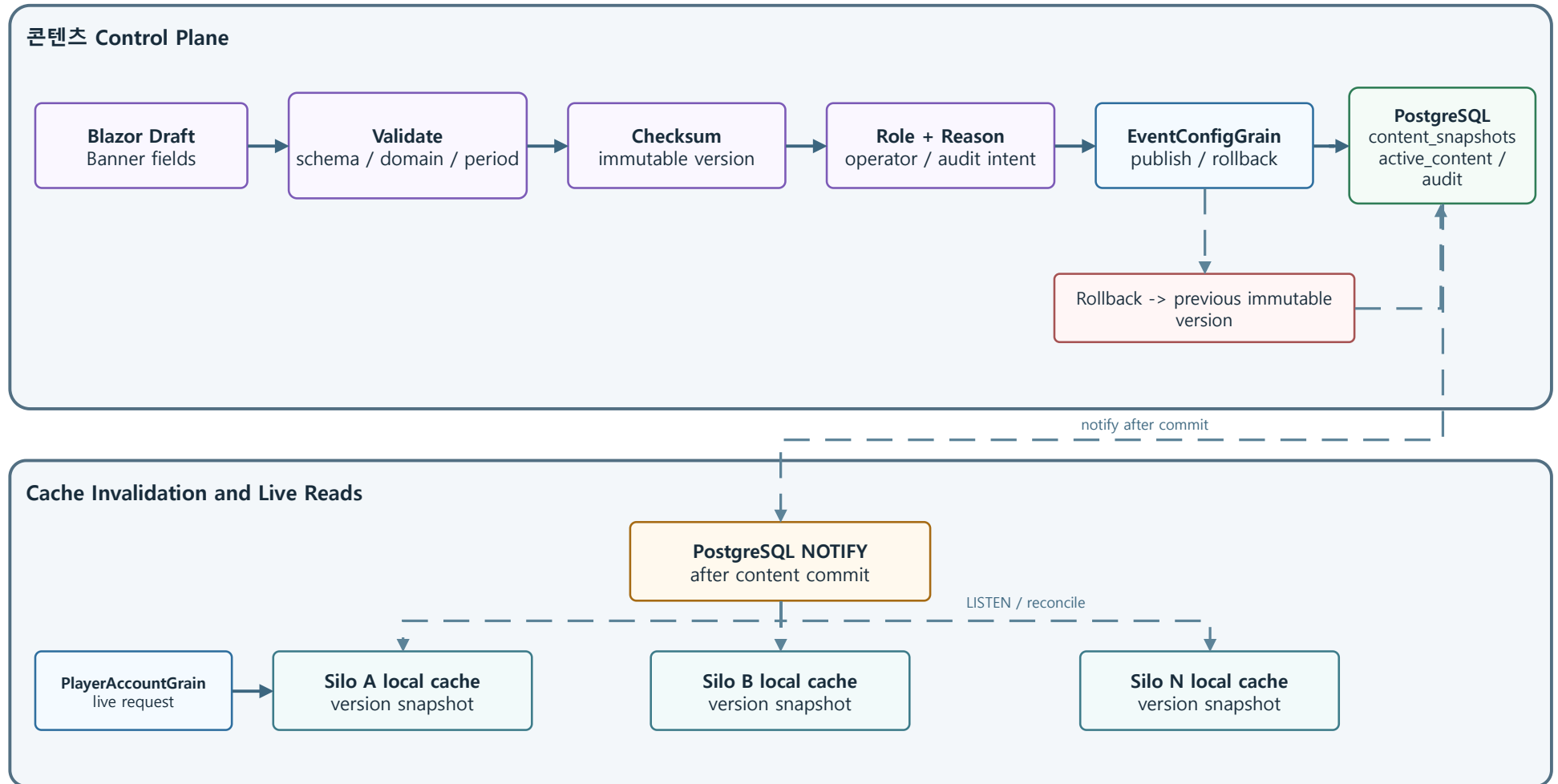
가차 결과, 재화, 인벤토리, 천장, 이력, Outbox, 먹등성 응답을 하나의 commit으로 묶는다.



Completed replay returns before reading current content; request idempotency stores no PENDING lock.

8.4 콘텐츠 배포와 Silo 캐시

EventConfigGrain은 control plane에만 두고, live request는 각 Silo의 local snapshot을 읽는다.

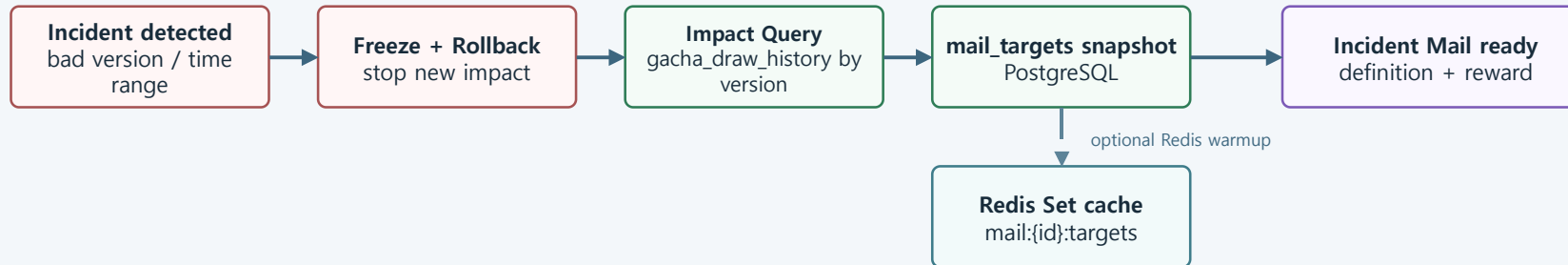


Hot path: PlayerAccountGrain -> local cache. It does not call EventConfigGrain for every request.

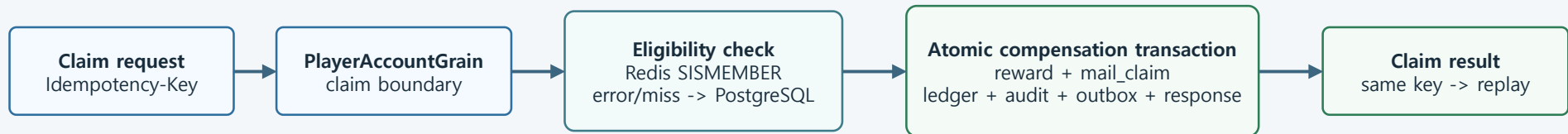
8.5 사고 복구와 보상

잘못된 콘텐츠를 되돌리는 것과 이미 영향을 받은 플레이어를 보상하는 것을 별도 단계로 처리한다.

운영 사고 대응



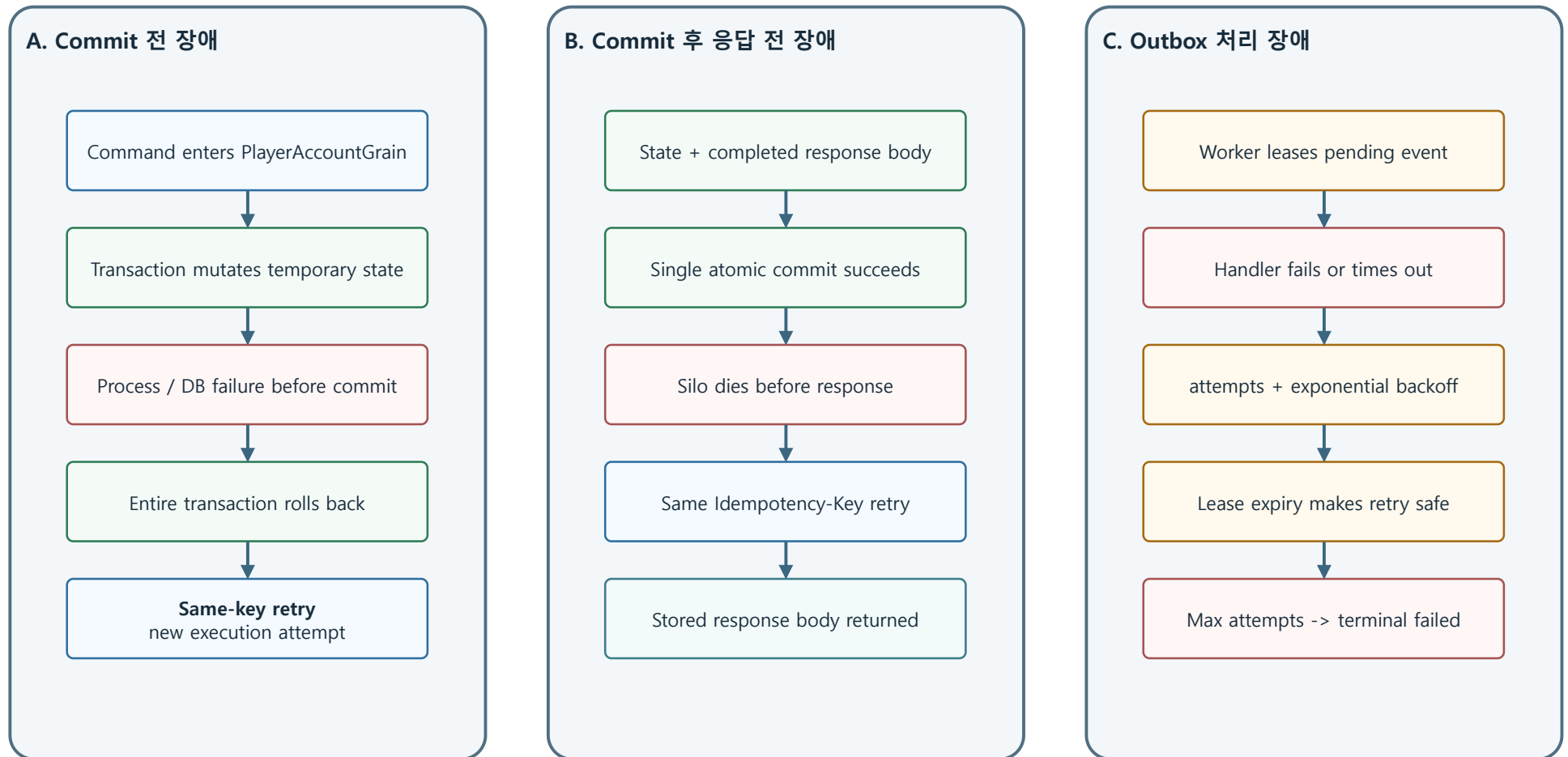
플레이어 보상 수령



Redis is never the compensation source of truth; connection failure must fall back to PostgreSQL.

8.6 멍등성과 장애 복구

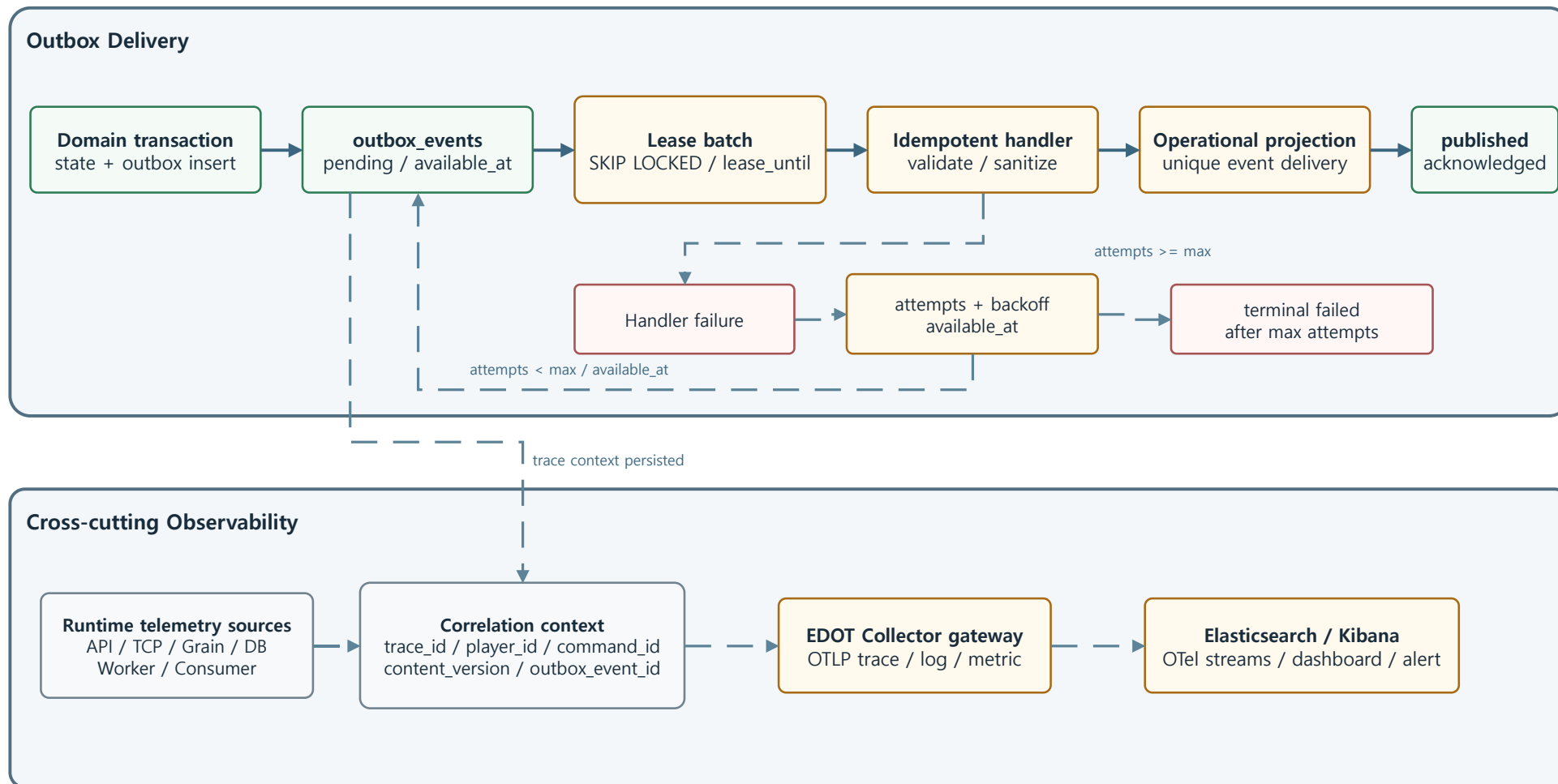
장애 시점을 분리해 rollback, response replay, Outbox retry의 책임을 명확히 한다.



Request idempotency: COMPLETED only, no PENDING | Outbox processing state: pending / processing / published / failed

8.7 Outbox와 관측성

Outbox는 idempotent consumer로 전달하고, Observability는 모든 실행 단계를 횡단해 관찰한다.



Persist trace context with the Outbox event so Worker spans can be linked to the originating command.

핵심 설계 결정

ADR-001. Orleans를 쓰는 이유

Orleans는 DB를 대체하기 위해 쓰지 않는다. ASTRA에서 Orleans의 역할은 플레이어별 명령 실행 경계다.

결정:

- 플레이어 상태 변경 명령은 PlayerAccountGrain을 통과한다.
- Grain은 동일 플레이어 command를 순차 처리한다.
- DB는 source of truth로 유지한다.
- 정합성은 Grain 직렬화 + DB transaction + DB constraint로 방어한다.

선택 근거:

단순 CRUD API로도 만들 수 있지만, 수집형 RPG 서버는 같은 유저에게 재화 사용, 보상 수령, 가차 요청이 거의 동시에 들어옵니다. Orleans를 사용하면 플레이어 단위 실행 경계를 명확히 만들 수 있고, DB는 최종 원장과 복구 기준으로 유지할 수 있습니다.

ADR-002. PlayerAccountGrain은 hot/cold state를 분리한다

위험:

- 수집형 RPG 인벤토리는 수백, 수천 row로 커질 수 있다.
- 모든 인벤토리와 우편 목록을 Grain state에 올리면 메모리와 serialization 비용이 커진다.

결정:

- Grain hot state: wallet summary, pity counters, daily counters, version, checksum, short command state
- DB cold state: inventory detail, mailbox list, gacha history, audit, ledger, claim history
- 조회는 DB paging
- 변경은 PlayerAccountGrain 직렬화 후 DB transaction

InventoryGrain 분리는 이후의 확장 선택지다. 현재 범위에서는 원자적 보상 처리와 명확한 transaction 경계를 위해 PlayerAccountGrain aggregate를 우선한다.

ADR-003. Idempotency PENDING을 제거한다

폐기한 기존 방식:

```
pending-first lock row 생성
-> domain processing
-> COMPLETED update
```

문제:

- PENDING commit 직후 Silo가 죽으면 고아 row가 남는다.
- 클라이언트 재시도가 PENDING에 막힐 수 있다.
- PlayerAccountGrain이 이미 player 단위 race를 줄이므로 요청 idempotency를 distributed lock처럼 쓸 이유가 약하다.

최종 결정:

- IDEMPOTENCY_REQUESTS에는 완료 결과만 저장한다.
- 도메인 변경과 completed response_body 저장은 같은 DB transaction에서 commit한다.
- commit 전 장애는 전체 rollback된다.
- commit 후 응답 전 장애는 재시도에서 기존 response_body를 반환한다.
- request idempotency table에는 일반 처리 흐름용 status column을 두지 않는다.

최종 schema 방향:

```
idempotency_requests(  
  player_id,  
  idempotency_key,  
  request_hash,  
  response_body,  
  expires_at,  
  completed_at,  
  primary key(player_id, idempotency_key)  
)
```

테이블 상태:

- request idempotency: COMPLETED 중심. FAILED/PENDING을 일반 흐름에 두지 않는다.
- outbox: PENDING/PROCESSING/PROCESSED/POISON 유지.
- compensation command: PENDING/APPLIED/FAILED 유지 가능.

ADR-004. response_body + TTL을 저장한다

result_ref만 저장하고 결과를 재조립하면 과거 데이터 변경, join 실패, schema drift에 취약하다.

결정:

- 성공 응답 response_body를 JSONB로 저장한다.
- TTL은 24시간 또는 서비스 정책에 맞춘다.
- 장기 이력은 gacha_draw_history, ledger_entries, admin_operation_audit에 남긴다.
- response_body는 재시도 응답용 단기 캐시로만 사용한다.

ADR-005. TCP Gateway는 API 뒤로 가지 않는다

폐기한 구조. 구현 금지:

TCP Gateway -> HTTP API -> Orleans

최종 구조:

HTTP API -> Orleans Client -> Grain
TCP Gateway -> Orleans Client -> Grain

이유:

- TCP의 transport 이점을 API 변환으로 잃지 않는다.
- HTTP와 TCP 모두 같은 command package를 공유한다.
- TCP Gateway는 session, framing, request id, reconnect를 책임진다.

ADR-006. 사고 보상 Mail 대상자는 snapshot으로 확정한다

문제:

사고 보상 대상이 "특정 시간에 특정 배너를 돌린 유저"라면 claim 시점마다 로그 전체를 조회하면 안 된다.

결정:

- incident 생성 후 batch가 대상자를 산정한다.
- PostgreSQL mail_targets(mail_id, player_id)에 영속 snapshot 저장.
- Redis Set mail:{mail_id}:targets는 SISEMEMBER 가속용으로만 사용.
- Redis 장애 또는 miss 시 PostgreSQL fallback.
- claim은 PlayerAccountGrain 경로에서 idempotent하게 처리한다.

ADR-007. EventConfigGrain은 hot path에 두지 않는다

문제:

모든 가차 요청이 EventConfigGrain을 직접 조회하면 이벤트 오픈 직후 hot spot이 된다.

결정:

- EventConfigGrain은 publish, rollback, cache invalidation의 control plane이다.
- 요청 hot path는 Silo-local ActiveContentCache를 우선 사용한다.
- PostgreSQL snapshot store와 periodic reconciliation이 복구 경로다.
- 처리 당시 content_version과 checksum을 audit에 남긴다.

ADR-008. PostgreSQL transaction은 정합성의 마지막 방어선이다

가차 transaction에 포함되는 것:

- wallet debit
- gacha_draw insert
- inventory upsert
- duplicate reward conversion
- pity update
- ledger insert
- audit insert
- outbox insert
- idempotency completed response_body insert

이 중 하나라도 실패하면 전체 rollback한다.

ADR-009. Outbox polling은 MVP에서는 허용하되 부하를 제한한다

MVP에서는 polling 기반 Outbox를 구현한다.

방어 장치:

- batch size 제한
- lease timeout
- retry count
- poison queue
- pending index
- exponential backoff

CDC/WAL은 현재 범위를 넘기므로 ADR에 확장 후보로 남긴다.

ADR-010. 시간 정책

- 모든 저장 기준은 UTC.
- 운영 UI는 KST 표시.
- content publish window는 UTC timestamp로 저장.
- 경계값 테스트를 둔다.
- DB server time 또는 injectable clock을 기준으로 일관성을 맞춘다.

구현 결과와 검증 기준

1. 솔루션 구조

```
astraliveops/  
src/  
  Astra.Api/  
  Astra.TcpGateway/  
  Astra.Silo/  
  Astra.Admin/  
  Astra.Worker/  
  Astra.Contracts/  
  Astra.Domain/  
  Astra.Infrastructure/  
tests/  
  Astra.UnitTests/  
  Astra.IntegrationTests/  
  Astra.ConcurrencyTests/  
  Astra.FailureTests/  
deploy/  
  docker-compose.yml  
helm/  
  pulumi/  
  terraform/  
docs/  
  adr/  
  api/  
  tcp/  
  operations/
```

2. Phase 0 - 요구사항 고정

산출물:

- 핵심 시나리오 2개 확정
- 기술 구성과 책임 범위
- 핵심 ADR 기준

완료 기준:

- "모든 콘텐츠 구현"이 아니라 "가차 정합성 + 운영 복구"로 범위가 잠김.

3. Phase 1 - .NET/Orleans 기반

구현:

- .NET 10 solution
- ASP.NET Core API
- Orleans Silo
- Orleans Client 연결
- PostgreSQL ADO.NET membership과 공식 10.2.1 schema migration
- PostgreSQL/Redis Docker Compose
- 기본 health check

검증:

- API에서 Grain 호출 성공
- Silo 재시작 후 Grain 재활성화 확인
- 같은 membership DB에 Silo 2개가 Active로 합류
- ADO.NET gateway discovery를 통한 HTTP/TCP E2E 4건 통과

4. Phase 2 - PlayerAccountGrain과 재화

구현:

- PlayerAccountGrain

- wallet debit/credit
- ledger
- audit
- DB transaction boundary

검증:

- 동시 차감 요청에서 음수 잔액 0건
- ledger 합계와 wallet 잔액 일치

5. Phase 3 - Idempotency 최종 구조

구현:

- request_hash
- completed response_body
- TTL cleanup
- same key replay
- conflict detection
- PENDING 제거

검증:

- 같은 Idempotency-Key 100회 재시도 시 실제 처리 1회
- 같은 key + 다른 body는 conflict
- commit 후 응답 전 장애를 재현하고 replay 성공

6. Phase 4 - 가차/천장/인벤

구현:

- gacha pool
- pickup table
- weighted draw
- pity state
- duplicate conversion
- inventory upsert
- gacha history

검증:

- 재화 차감, 지급, 천장 갱신이 한 transaction
- 장애 주입 시 부분 지급 0건

7. Phase 5 - 콘텐츠 버전과 Admin

구현:

- content draft input
- validation result
- immutable checksum
- role/reason audit
- rollback
- ActiveContentCache

- EventConfigGrain invalidation
- 개발용 loopback 로그인과 선택형 운영 OIDC/BFF 인증

검증:

- 잘못된 확률표 validation 실패
- publish 후 cache 갱신
- rollback 후 신규 요청은 이전 안정 버전 사용
- OIDC authority 모드에서 로컬 API signing key 없이 JWT 검증 구성
- 외부 access token은 브라우저 쿠키에서 제거하고 pod-local session에만 보관

8. Phase 6 - Incident와 대상자 Mail

구현:

- incident 생성
- impact query
- mail_targets snapshot
- Redis Set cache
- DB fallback
- mail claim
- compensation ledger/audit

검증:

- 특정 content_version 대상자만 보상 수령 가능
- Redis 장애 시 DB fallback
- 중복 claim 0건

9. Phase 7 - TCP + Protobuf

구현:

- length-prefix framing
- protobuf command envelope
- request_id
- idempotency_key
- session/player binding
- Orleans Client 직접 호출

검증:

- HTTP와 TCP가 같은 command 결과 생성
- TCP retry/reconnect에서 idempotency replay 동작

10. Phase 8 - Observability

구현:

- OpenTelemetry trace/log/metric과 delta Counter
- EDOT Collector gateway
- OTel-native Elasticsearch data stream
- Kibana Dashboard as Code와 alert rule
- 1 GiB Elasticsearch tmpfs, 1시간 retention, 회전 로그

- 실제 pool saturation과 Outbox fault injection probe

검증:

- API/TCP/Silo/Worker service trace 수집
- acquisition attempts/failures 기반 99.9% SLO burn rate 계산
- 실제 2-slot 포화: attempts 4, timeout 1, recovery 1, burn rate 250x
- 실제 Outbox 처리: published 1, retry 1, dead-letter 1, alert active
- dashboard 7개 panel Playwright 시각 검수

11. Phase 9 - CI/CD와 Infra

구현:

- GitHub Actions build/test와 deployment-validation job
- API/Admin/Silo/TCP Gateway/Worker multi-target Dockerfile
- migration hook, PDB, probes, resource/security limits를 포함한 Helm chart
- private AKS/ACR/VNet/Log Analytics/private PostgreSQL Terraform foundation
- 기존 AKS에 Helm release만 배포하는 Pulumi C# program

검증:

- PR에서 PostgreSQL 및 ADO.NET Orleans E2E 자동 실행
- CI에서 Docker image 5종 build
- Helm lint/render 11 resources 통과
- Terraform azurerm 4.80 validate 통과
- Pulumi C# build 0 warning/0 error, offline preview 3 resources
- 실제 Azure plan/apply와 Pulumi up은 자격증명/비용 경계로 분리

12. 최종 데모 스크립트

pwsh -File scripts/demo/Run-PortfolioDemo.ps1 한 명령으로 다음을 실행한다.

1. PostgreSQL만 필요 시 기동하고 Release build
2. Silo/API/TCP/Admin/Worker를 ADO.NET membership으로 기동
3. 기존 E2E로 HTTP/TCP cross-transport replay 확인
4. 픽업 content publish와 active checksum 확인
5. 500 엘리프 지급 후 100 엘리프 가차를 같은 key로 재시도
6. 실제 차감 1회, 동일 response replay, content version/checksum 확인
7. 영향 유저 snapshot 기반 200 엘리프 incident mail 생성
8. 같은 claim key 재시도 후 실제 보상 1회와 최종 잔액 600 확인
9. Redis 비활성 상태에서 PostgreSQL target fallback 확인
10. audit 필수 action과 Outbox published delta 확인
11. 토큰을 제외한 최신 JSON/Markdown/TCP 로그 3개를 고정 경로에 덮어쓰기

Admin은 실행 상태로 남아 결과를 화면에서 확인할 수 있다. 별도 **Test-OperationalScenarios.ps1**은 PostgreSQL pool saturation과 Outbox invalid payload를 주입해 Kibana burn rate/dead-letter alert 및 JSON evidence를 검증한다.

13. 완료 기준

영역	기준
정합성	중복 지급, 음수 잔액, 부분 지급 0건
복구	commit 후 응답 장애에서 replay 성공
운영	Admin publish/rollback/compensation 시연
성능	PostgreSQL pool 포화/회복과 TCP delivery latency 계측
관측	실제 포화/실패를 trace, burn rate, Outbox metric, alert로 재현
기술	주요 구성요소를 동작 산출물로 검증

14. 검증 산출물

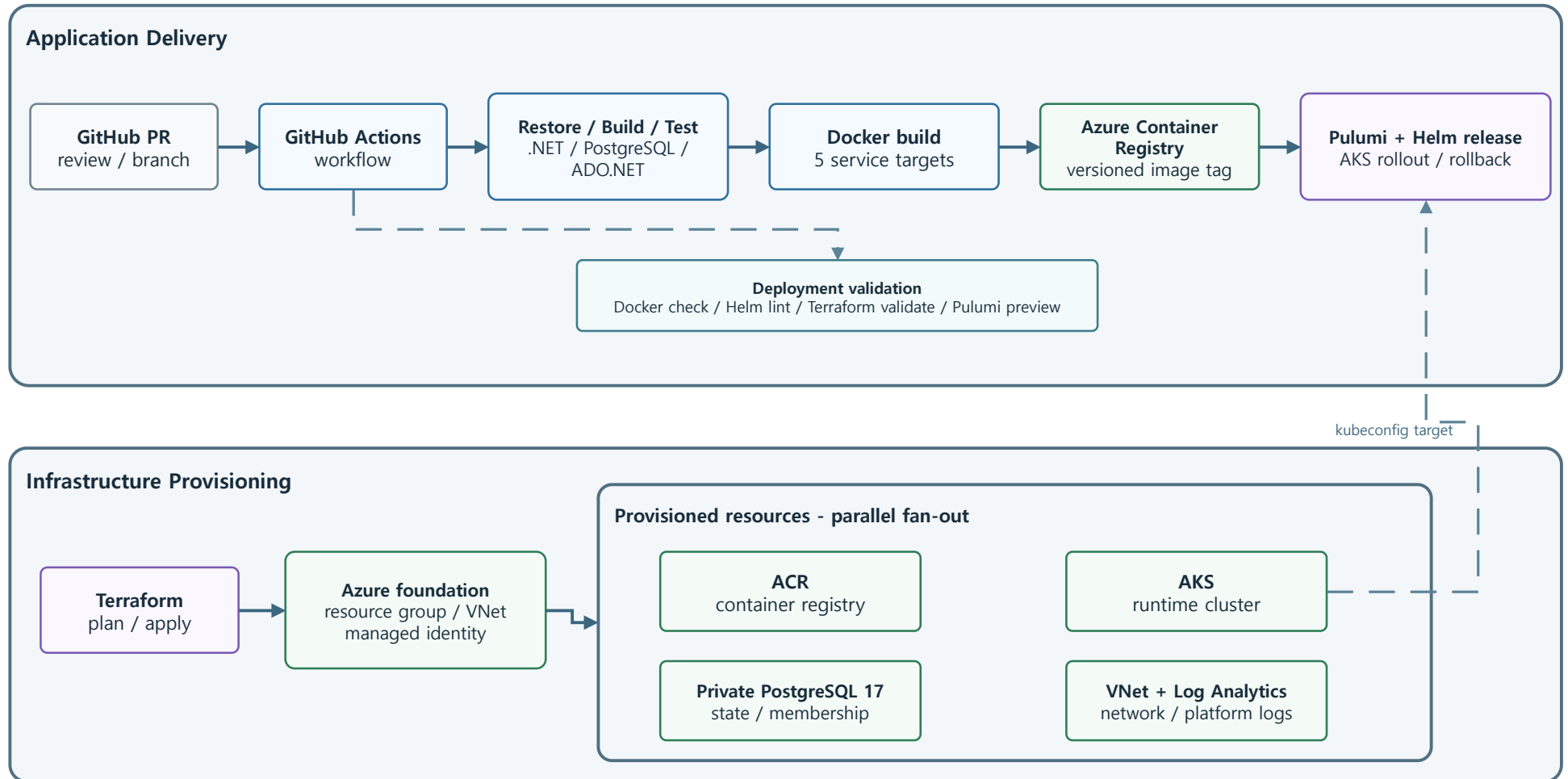
- **scripts/demo/Run-PortfolioDemo.ps1**: 핵심 시나리오 한 명령 재현
- **output/demo**: 실행 결과 JSON, 요약 Markdown, TCP E2E 로그
- **output/playwright**: Blazor Admin 및 Kibana 화면 증빙
- **.github/workflows/ci.yml**: build/test와 배포 구성 검증

15. 검증 경계

- 실제 Azure plan/apply와 외부 IdP tenant 연동은 자격증명 및 비용 경계 밖이다.
- Redis는 사고 우편 대상 membership cache로 사용하며 오류 또는 miss 시 PostgreSQL로 fallback한다.
- 전 유저 공지 우편, 랭킹과 CDC/WAL은 확장 범위다.

8.8 CI/CD와 Azure 배포

런타임 요청 흐름과 배포/IaC 흐름을 분리해 배포 책임과 검증 경계를 명확하게 보여준다.

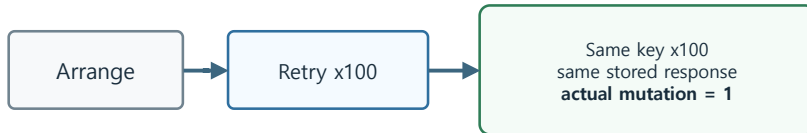


Local: Docker Compose | CI: GitHub Actions | Registry: ACR | Runtime: AKS | IaC: Terraform | App release: Pulumi + Helm

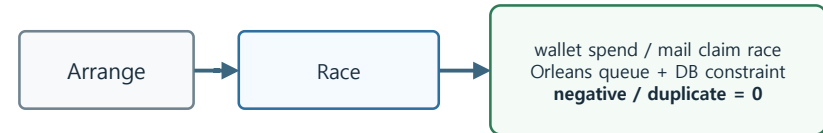
8.9 핵심 검증 시나리오

아키텍처 주장을 기능 개수 대신 실패·동시성·복구 테스트로 증명한다.

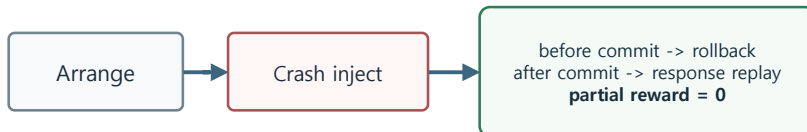
1. Idempotency Replay



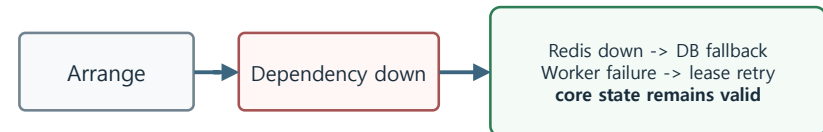
2. Concurrent Player Commands



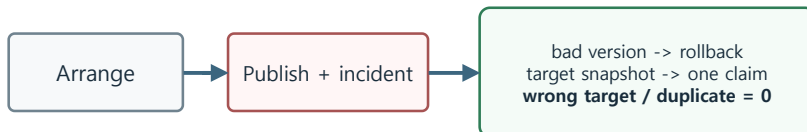
3. Crash Windows



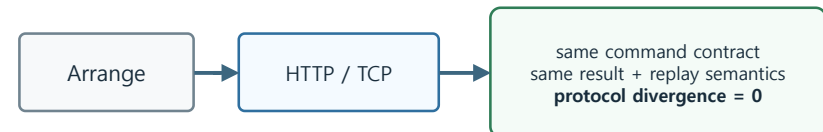
4. Dependency Failure



5. Content Rollback + Compensation



6. HTTP / TCP Command Parity



Verification evidence - xUnit 91 / PostgreSQL 17 / ADO.NET 2-Silo + E2E 4 / Docker 5 targets / Helm lint / Terraform validate / Pulumi preview

Tests must fail visibly when dependencies are not available; gated tests must not silently return as passed.